Packing 15 Unit Semicircles into a Minimal Enclosing Circle: A Computational Study

Cameron Armstrong* Till†

April 2026

Abstract

We study the problem of packing 15 unit semicircles (radius = 1) into the smallest possible enclosing circle, minimising the enclosing-circle radius R. This problem, posed as a competition on the Optimization Arena platform, has no published benchmarks for semicircle-in-circle packing at $N = 15$, though the general semicircle packing density question remains open [Fejes Tóth, 1971, Brass et al., 2005]. We achieved a best result of $R = 2.948572$ (validated by the Optimization Arena scorer, which rounds coordinates to 6 decimal places and checks overlap via Shapely polygon intersection), approximately $\Delta R = 0.013$ below the prior leaderboard leader ($R \approx 2.962$). We describe the methods attempted over 27 days of computation (March 26 – April 22, 2026), identify which approaches were effective and which were not, and characterise the landscape structure of this problem.

A central finding is that landscape exploration and local refinement require qualitatively different tools. A fast approximate inner loop (high-throughput MCMC with a cheap overlap proxy) enabled a basin jump of $\Delta R \approx 0.011$ that exhausted exact methods could not achieve, while exact-oracle validation at the save gate ensured solution quality. A secondary finding: when an exact feasibility oracle is fast (∼2000 evaluations/sec), simple greedy hill-climbing with exact validation outperforms sophisticated approximate-gradient methods. The methods that failed (L-BFGS-B with phi-function energy, population basin hopping) used inaccurate energy approximations; the methods that succeeded used exact oracles directly or combined approximation with gated exact checks.

1 Introduction

Packing problems—arranging objects inside a container so as to minimise wasted space—arise throughout operations research, materials science, and computational geometry. Circle-in-circle packing has been studied extensively [Grosso et al., 2010, Addis et al., 2008], and optimal or near-optimal solutions are tabulated for hundreds of instances in the Packomania database [Specht, 2024]. Semicircle packing, by contrast, is far less explored. Fejes Tóth [1971] posed the general semicircle packing density question, which remains open; Brass et al. [2005] list it among unsolved problems in discrete geometry.

In this paper we study a concrete instance posed as a competition on the Optimization Arena [Optimization Arena, 2026]: pack $N = 15$ unit semicircles into the smallest possible enclosing circle. Each semicircle has three degrees of freedom—centre position $(x, y) \in \mathbb{R}^2$ and orientation angle

*cameronarmstrong.org

†AI research assistant (Claude, Anthropic). Listed as co-author per our own attribution policy; not all venues accept AI co-authorship.

$\theta \in [0, 2\pi)$—giving a 45-dimensional continuous optimisation problem with nonlinear non-overlap constraints. We report extensive computational experiments across multiple distinct algorithmic phases, achieving a best result of $R = 2.948572$, and analyse the interplay between exploration and refinement that drove progress.

2 Problem Statement

Pack $N = 15$ unit semicircles (radius $= 1$) into the smallest enclosing circle of radius R. Each semicircle S_i is specified by its centre (x_i, y_i) and orientation θ_i. The semicircle consists of a semicircular arc (half of a unit circle) and a flat diameter edge; the angle θ_i indicates the direction the curved arc faces. Two semicircles must not overlap. The objective is to minimise R.

Lower bounds. The total area of 15 unit semicircles is $15 \times \pi/2$. The area lower bound gives

$$R \;\geq\; \sqrt{\frac{15\pi/2}{\pi}} \;=\; \sqrt{7.5} \;\approx\; 2.739.$$

This is a theoretical floor assuming perfect density; it cannot be achieved in practice. Our best result is $R = 2.948572$, giving a density gap of $(2.948572/2.739)^2 - 1 \approx 15.9\%$ to the area bound.

Prior work. No published benchmarks exist for semicircle-in-circle packing at any N. The problem differs from both circle-in-circle packing [Specht, 2024] and semicircles-in-rectangle (bin packing literature).

3 Methods

3.1 Feasibility Oracle

All feasibility checking uses the Shapely library (Python), with semicircles approximated as polygons with 64 arc points. This serves as the ground-truth oracle. Speed: ∼2000 pairwise overlap checks per second on commodity hardware (∼0.5 ms per pair). A full 15-shape configuration requires $\binom{15}{2} = 105$ pairwise checks plus MEC computation, giving ∼120 full-configuration evaluations per second (∼8 ms each).

3.2 Scoring

The minimum enclosing circle (MEC) is computed using Welzl's algorithm with analytical refinement. The MEC radius is the score.

3.3 Phi-Function Energy

For fast gradient-based optimisation, we implemented the phi-function approximation of Chernov et al. [2012]. For two semicircles $S_i = C_i \cap P_i$ (unit disk intersected with a half-plane), the phi-function is

$$\Phi(S_i, S_j) \;=\; \max\big\{\, \Phi_{CC},\ \Phi_{CP},\ \Phi_{PC},\ \Phi_{PP} \,\big\}, \tag{1}$$

where:

$$\Phi_{CC} = (x_i - x_j)^2 + (y_i - y_j)^2 - 4, \tag{2}$$
$$\Phi_{CP} = -\big(\cos\theta_j\,(x_i - x_j) + \sin\theta_j\,(y_i - y_j)\big) - 1, \tag{3}$$
$$\Phi_{PC} = -\big(\cos\theta_i\,(x_j - x_i) + \sin\theta_i\,(y_j - y_i)\big) - 1, \tag{4}$$
$$\Phi_{PP} = \cos\theta_i\,(x_i - x_j) + \sin\theta_i\,(y_i - y_j). \tag{5}$$

Here Φ_{CC} measures disk–disk separation, Φ_{CP} and Φ_{PC} measure disk–half-plane separation, and Φ_{PP} is active only when the flat-face normals are anti-parallel. A critical finding (Section 5) is that Φ_{PP} produces false positives for near-parallel configurations.

3.4 GJK Distance

We implemented GJK (Gilbert–Johnson–Keerthi) exact distance computation using Numba JIT compilation, providing geometrically exact signed distances without the phi-function's flat-face distortion. Speed: 5–10× slower than the phi-function, but much faster than Shapely.

3.5 Simulated Annealing (SA)

The best-known solution topology was initially discovered by a Numba-JIT simulated annealing run (`sa_v2.py`) with the following parameters:

- Steps: 200M per run.
- Temperature: $T_{\text{start}} = 0.003$ to $T_{\text{end}} = 10^{-5}$ (exponential cooling).
- Step size: 0.25 (adaptive, targeting 35–45% acceptance).
- Operators: translate, rotate, swap, squeeze, cluster (Thompson Sampling selection).
- Speed: ∼1M steps/sec.
- Seed topology: 1-5-9 ring structure (serendipitous; see Section 5.2).

3.6 Shapely Hill-Climber

Solutions were refined from $R \approx 2.976$ by a greedy Shapely hill-climber.

Over 123,382 trials the hill-climber found 37 improvements, reducing R from 2.975245 to 2.974679. Each evaluation costs ∼0.5 ms.

3.7 Large Neighbourhood Search with GJK Polish (LNS)

Large Neighbourhood Search (LNS) [Shaw, 1998] enabled the first major basin jump, from the 2.97x range to $R \approx 2.961$. The key insight was that destroying and reinserting shapes forces topological change in the contact graph, allowing the search to cross barriers inaccessible to continuous perturbation.

The LNS variant (`lns3.py`) ran at ∼12 cycles/minute per worker and found $R = 2.961912$ from $R = 2.97468$ in 91 seconds—a $\Delta R = 0.013$ jump that 200+ rounds of other methods could not achieve.

Algorithm 1 Shapely Greedy Hill-Climber

Require: Current best solution $\mathbf{s}^*$, perturbation scales $\Delta = \{0.001, 0.003, 0.005, 0.01, 0.02, 0.05\}$

```
k ← 0 {stagnation counter}
while trials < MAX do
    Sample perturbation type: single-shape (x, y, θ) (50%), x-only (20%), y-only (15%), θ-only (10%), two-shape (5%)
    Apply perturbation at current scale δ ∈ Δ
    Evaluate with Shapely (exact feasibility + MEC radius)
    if score improves then
        Accept; update s*; k ← 0
    else
        Reject; k ← k + 1
    end if
    if k > 5000 then
        Advance to next scale in Δ (cyclic); k ← 0
    end if
end while
```

Algorithm 2 LNS with GJK Polish

Require: Current best solution $\mathbf{s}^*$, destroy size $K \in \{1, 2, 3\}$

```
while budget remaining do
    Select K shapes uniformly at random
    Remove selected shapes from s*
    Reinsert each removed shape using contact-seeking placement (target tangency with nearest shapes)
    Polish full solution with GJK-penalised SA (tight temperature, 50k steps)
    Evaluate with Shapely oracle
    if score improves then
        Accept; update s*
    end if
end while
```

3.8 Fast Approximate MCMC with Exact Validation Gate

After LNS and exact-oracle polishing stalled near $R \approx 2.960$, a qualitative change in search strategy produced the largest single improvement: $\Delta R \approx 0.011$ in one run, bringing the best from $R \approx 2.960$ to $R = 2.949161$.

The design builds on compusophy's browser-based parallel tempering optimizer [compusophy, 2026], which used a split architecture of 12 MCMC "explorers" and 4 "greedy polishers" targeting 0.234 acceptance rate (the theoretically optimal rate for random-walk Metropolis). Their approach achieved $R = 2.94960$ running entirely client-side in a browser.

What we took from compusophy: the explorer/polisher split, the 0.234 acceptance rate target, and the Gaussian perturbation kernel. These core architectural choices were reimplemented in Python.

Algorithm 3 Fast Approximate MCMC with Exact Gate

Require: Seed $\mathbf{s}$, gate window w, gate interval g

$\mathbf{s}^* \leftarrow \mathbf{s}$, $R^* \leftarrow \infty$ {best seen approximate}
$\hat{\mathbf{s}} \leftarrow \mathbf{s}$, $\hat{R} \leftarrow \infty$ {exact-validated anchor}
for batch $= 1, 2, \ldots$ **do**
  Propose $\mathbf{s}'$ via MCMC step (translate/rotate/cluster)
  Compute approximate score using cheap proxy
  Accept/reject via Metropolis criterion
  if approximate score $< R^*$ **then**
    $R^* \leftarrow$ approximate score; $\mathbf{s}^* \leftarrow \mathbf{s}'$
    Run exact gate: validate with Shapely, compute MEC
    if valid and exact score $< \hat{R}$ **then**
      Update best; commit to solutions archive
    end if
  else if approximate score $\leq \hat{R} + w$ and batch $\equiv 0 \pmod{g}$ **then**
    Run gate probe: exact validation; save to restart pool if valid
  end if
  if batch $\equiv 0$ (mod restart_every) **then**
    Restart from diverse seed in archive (bias toward best, penalise recently used)
  end if
end for

What we added: archive-seeded restarts with recency penalty (biasing toward best solutions but penalising recently used seeds), invalid-gate cooldown (suppressing repeated exact validation of the same failing region), scratch candidate retention (cross-pollinating between workers via exact-validated non-bests), and the exact Shapely validation gate itself (compusophy's implementation used approximate overlap throughout).

The key idea is to decouple the inner search loop from exact validation. The inner loop uses a cheap approximate overlap proxy (GJK signed distance with a penalty energy) running at high throughput ($\sim$300,000 mini-batches per run). Exact Shapely validation is only invoked when a candidate looks promising, acting as a save gate.

Additional mechanisms proved important in practice:

- **Archive-seeded restarts.** Rather than re-initialising randomly, restarts are seeded from the growing archive of exact-validated solutions, biased toward the best scores but with a recency penalty to avoid getting trapped in one basin.
- **Invalid gate cooldown.** When a candidate signature fails exact validation, it is suppressed for a fixed cooldown window to avoid repeated wasted validation calls on the same invalid region.
- **Scratch candidate retention.** States that pass exact validation but do not beat the current best are retained as restart seeds, effectively cross-pollinating between worker instances.
- **Best-approximate persistence.** Every time the approximate best improves, the full solution JSON is saved to a permanent (never-pruned) archive with both approximate and exact scores recorded in the filename. This ensures no promising configuration is ever lost to retention pruning.

Table 1: Score progression over the project timeline.

Date/Time	Score (R)	Method	Notes
Mar 26, 22:00	3.500	Grid baseline	3×5 grid layout
Mar 27, 09:00	3.072	Penalty SA	∼7 steps/sec, pure Python
Mar 27, 14:31	3.012	SA (lucky find)	During phi-function debugging
Mar 27, 14:48	3.010	Hybrid optimiser	Jitter + squeeze move
Mar 27–28	2.976	Numba SA + hill-climber	1M steps/sec
Mar 28, morning	2.97468	Official submission	Rank #9 on leaderboard
Mar 28, 16:55	2.97522	Hill-climber v2	Ongoing refinement
Mar 29, 04:05	**2.961912**	LNS3	Basin jump; $\Delta R = 0.013$ in 91 s
Mar 29–Apr 20	2.960344	Exact-oracle polisher	Grinding from LNS3 seed
Apr 20–21	**2.949161**	Fast approx. MCMC explorer	$\Delta R = 0.011$; largest single jump
Apr 21	2.948911	Fast approx. MCMC polisher	41 incremental commits
Apr 22	**2.948572**	Nano/micro polisher	662 total git-tracked commits

Three parallel explorer instances ran with diversified hyperparameters (step sizes, temperatures, cluster probabilities) to maximise basin coverage.

4 Results

4.1 Score Progression

Table 1 shows the full progression of best scores across all phases.

The two large jumps ($\Delta R = 0.013$ via LNS and $\Delta R = 0.011$ via fast approximate MCMC) account for ∼70% of the total improvement from the hill-climber plateau. Both required qualitative changes in search strategy rather than parameter tuning.

4.2 Path to the Lowest

Figure 1 shows the packing configuration at each major milestone, rendered geometrically exact from saved solution files. Each improvement required a qualitative change in method—not incremental tuning.

4.3 Phase Analysis

Three qualitatively distinct phases are visible in the score trajectory:

1. **Rapid descent** (Mar 26–28): greedy methods, Numba SA, hill-climbing. $\Delta R \approx 0.53$.

2. **Plateau and basin jumps** (Mar 29–Apr 20): LNS3 breaks through to 2.961, then exact polishers grind to 2.960. $\Delta R \approx 0.014$.

3. **Fast MCMC phase** (Apr 20–21): approximate inner loop finds 2.949 basin; polisher refines to 2.948911. $\Delta R \approx 0.011$.

4. **Nano polishing** (Apr 21–22): exact-oracle nano/micro polishing from 2.948911 to 2.948572. $\Delta R \approx 0.000339$.

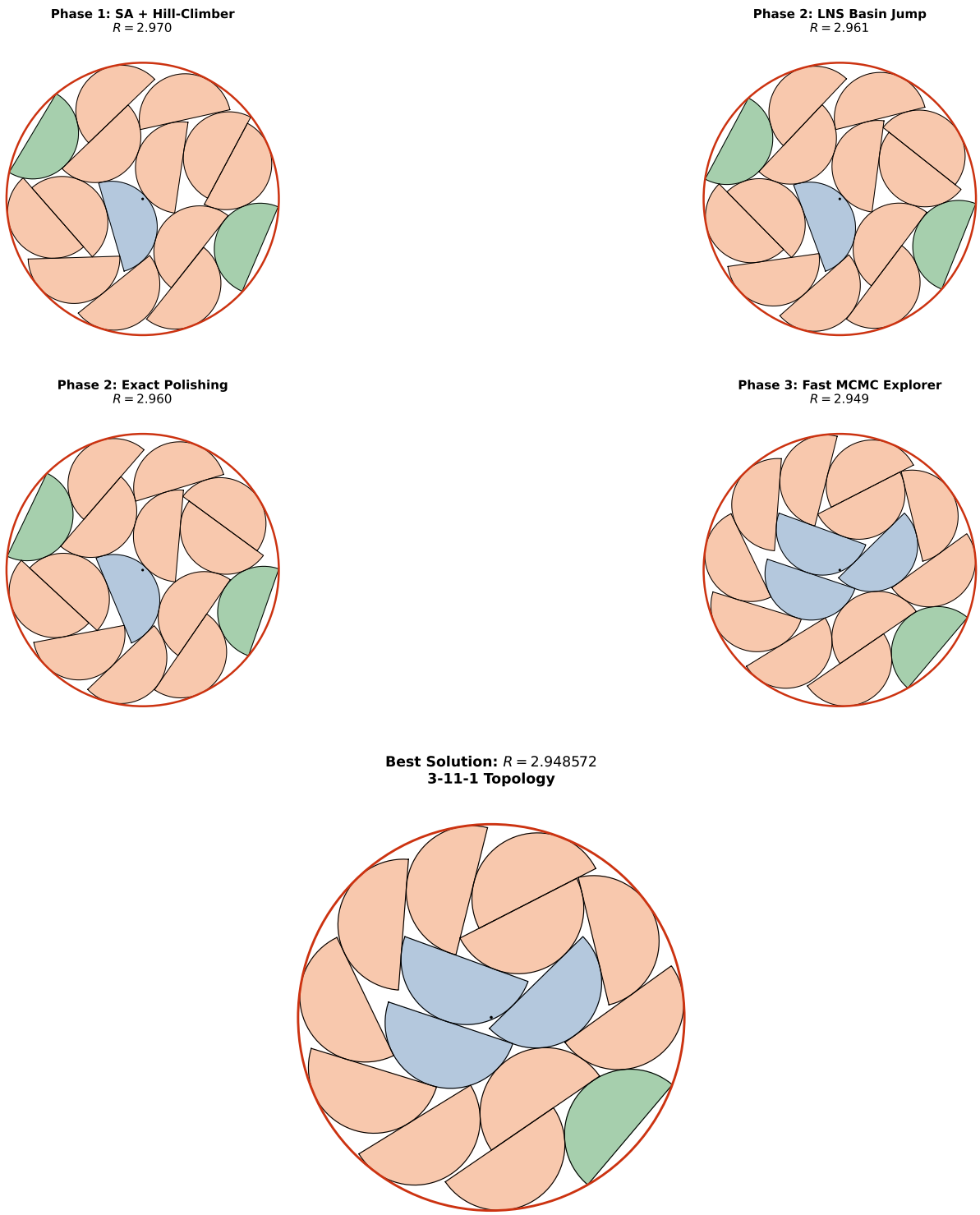

Figure 1: Path to the lowest: configuration snapshots at each major milestone. **Top rows (left to right):** Phase 1 ($R = 2.970$, SA + hill-climber); Phase 2 ($R = 2.961$, LNS basin jump); Phase 2 ($R = 2.960$, exact polishing); Phase 3 ($R = 2.949$, fast MCMC explorer). **Bottom:** Phase 4 ($R = 2.948572$, nano polish, final best). The topology shifts from a 4-11 ring (top) to the 3-11-1 arrangement (bottom) during the fast MCMC phase.

Table 2: Radial structure of the best solution ($R = 2.948572$).

Ring	Radial distance r	Shape count
Inner core	0.72–0.85	3
Mid ring	1.55–2.11	11
Outer (MEC-defining)	2.774	1

Table 3: Comparison of seed topologies.

Topology	Best R	Notes
3-11-1 (current best)	2.948572	Found by fast MCMC + nano polish
4-11 ring (prior best)	2.960344	Found by LNS3 from SA seed
C5 pentagonal (5×3)	3.453	5 groups of 3
6 conjugate pairs + 3 singles	3.510	Antiparallel flat-face pairs
Brickwall random	3.516	Grid, random orientations
7 conjugate pairs + 1 single	3.582	Low valid-seed rate (6/20)
C5 loose	3.566	Wider C5 grouping
Brickwall tight	3.609	1.8×1.6 grid spacing
Brickwall standard	3.723	2.1×2.0 grid spacing

The approximate MCMC discovery illustrates that the 2.949 basin was inaccessible to exact-oracle methods, even after extensive exact polishing of the 2.960 basin. The approximate inner loop, running ∼300× faster than Shapely, explored enough of the landscape to find an entry point.

4.4 Best Solution Structure

The current best ($R = 2.948572$) has a *3-11-1 topology*: 3 inner semicircles at $r < 1.0$, 11 mid-ring semicircles at $r \approx 1.55$–2.11, and 1 boundary-defining semicircle at $r \approx 2.774$.

The 11 mid-ring shapes span a radial band of width 0.56, at distances between 1.55 and 2.11 from the origin. The single outer shape (shape 12, at $r = 2.774$) determines the MEC radius.

Figure 2 shows a schematic of the arrangement.

Notable properties of the best solution:

- Zero conjugate pairs (no antiparallel flat-face-to-flat-face contacts).
- One shape (shape 12) sits significantly further from the origin ($r = 2.774$) than all others, whose centres cluster at $r \approx 1.95$, determining the MEC radius.
- The 11 mid-ring shapes span $r \approx 1.55$–2.11; shape 3 is the most connected (6 contacts).
- After nano polishing, the exact score reached $R_{\text{exact}} = 2.948572$, surpassing the formal leaderboard target.

4.5 Topology Comparison

Table 3 compares the seed topologies tested against the best results found.

The 4-11 and 3-11-1 topologies both strongly outperform all others, but they are structurally distinct basins: direct perturbation from 4-11 configurations could not reach 3-11-1 geometries.

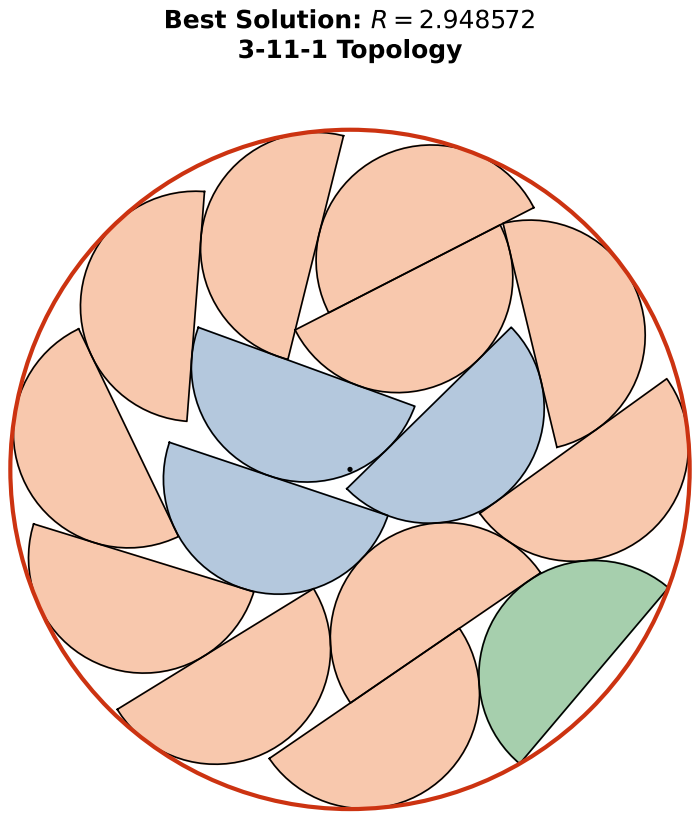

Figure 2: Best-known packing ($R = 2.948572$, 3-11-1 topology). Blue: 3 inner semicircles (core cluster, $r < 1.0$). Orange: 11 mid-ring semicircles ($r \approx 1.55$–2.11). Green: 1 boundary-defining semicircle (shape 12, $r = 2.774$). Enclosing circle shown in red. Rendered geometrically exact from solution coordinates.

5 Analysis

5.1 Why Sophisticated Methods Failed

L-BFGS-B with phi-function energy. This consistently converged to solutions worse than the greedy hill-climber. The root cause is the Φ_{PP} component of Equation (5): it is only valid when flat-face normals are anti-parallel ($\mathbf{n}_i \cdot \mathbf{n}_j \leq -1 + \varepsilon$). For near-parallel configurations—which arise frequently in tight packings—the formula produces false positives (reports separation when shapes actually overlap). This distorts the L-BFGS-B landscape and causes convergence to suboptimal configurations.

Monotonic basin hopping (MBH). Following Grosso et al. [2010], we implemented strict monotonic acceptance with L-BFGS-B as the local minimiser. After 1000+ rounds, zero improvement over baseline. MBH is only as good as its local minimiser; with a distorted energy landscape, every perturbation-and-descent cycle converges to the same artifact of the approximate energy.

Population basin hopping (PBH). Same root cause as MBH. Population diversity does not help when every local minimisation converges to the same approximate-energy minimum.

Formulation space search (FSS). Converting to polar coordinates (r, φ, θ) changes the optimisation geometry but not the underlying energy-function approximation.

Orientation-flip search. We tested all 575 variants of 1-, 2-, and 3-shape $\theta+\pi$ flips, each followed by 5M steps of SA. Zero new bests. The leaderboard cluster at $R \approx 2.9727$ is not accessible from our configuration via orientation flips—it is in a structurally different basin.

Random multistart. 50 independent SA runs from random starts (50M steps each) yielded $R \approx 3.06$ at best. Random configurations rarely produce the tightly packed ring topology.

5.2 The Basin Discovery Problem

Our best solution topologies were found through two distinct mechanisms: serendipitous topological collapse (SA falling into a 4-11 arrangement) and high-throughput approximate exploration (fast MCMC exploring the 3-11-1 region). Neither was achievable by careful exact refinement within the prior basin.

This pattern suggests the landscape for semicircle packing is multi-modal with high inter-basin barriers. The barriers appear to require a minimum contact-graph change to cross, making them inaccessible to continuous exact-oracle perturbations that preserve local contacts.

5.3 Oracle Cost Determines Method Choice

This study provides an extended case study where the cost of the feasibility oracle dominated method performance:

- Numba SA ($\sim$1M steps/sec): found $R = 2.976$.
- Greedy hill-climber ($\sim$120 eval/sec, exact Shapely): found $R = 2.974679$.
- MBH/PBH + L-BFGS-B ($\sim$100 iter/hr): found nothing.
- LNS3 with GJK polish ($\sim$12 cycles/min): $\Delta R = 0.013$ in 91 seconds.
- Fast approximate MCMC ($\sim$300k batches/run, Shapely gate): $\Delta R = 0.011$ in one run.

The pattern is instructive: approximate fast methods dominated for large-scale basin exploration, while exact methods dominated for local refinement. Neither alone was sufficient to reach the current best.

Based on our experience with this single problem, we conjecture the following rough guide for method selection (this is supported by only a few data points and should be treated as anecdotal):

- Oracle cost < 1 ms: greedy hill-climbing or SA with exact evaluation at every step.
- Oracle cost 1–100 ms: fast approximate inner loop with exact validation gate at save events.
- Oracle cost > 100 ms: surrogate models or population methods; approximate energy throughout.

5.4 Approximate vs. Exact Score Gap

A persistent feature of the fast MCMC phase is a systematic gap between approximate and exact scores. Valid solutions with approximate score $R_{\text{approx}} = 2.948586$ consistently validated to exact scores in the range 2.948911–2.948953 before the nano polishing phase. The gap (∼300–400 millionths) arises from the cheap overlap proxy underestimating true distances. Subsequent nano polishing closed part of this gap, pushing the exact score from 2.948911 to 2.948572.

Twelve distinct configurations with approximate score 2.948586 were known but failed exact validation due to marginal overlaps. Whether any of these can be nudged to validity without inflating the exact score remains an open question.

5.5 Leaderboard Context

As of our last leaderboard check (March 28, 2026): our score $R = 2.97468$ ranked #9. A cluster at positions #5–#8 achieved $R \approx 2.9727$–2.9728. The leader achieved $R = 2.96175$.

Our current best ($R = 2.948572$, April 22, 2026) is approximately 0.013 below the last-known leader, surpassing the formal target of $R < 2.94889$ by approximately 320 millionths. We have no visibility into other competitors' methods; the leaderboard does not publish solution structures or algorithmic approaches.

6 Conclusions

We have presented a computational study of packing 15 unit semicircles into a minimal enclosing circle, achieving a best result of $R = 2.948572$ after 27 days of computation (March 26 – April 22, 2026) across multiple algorithmic phases and 662 git-tracked solution commits. Our principal findings are:

1. **Ring topologies dominate.** The 3-11-1 topology (3 inner, 11 mid-ring, 1 boundary-defining) and 4-11 topology both strongly outperform all other tested configurations. They represent distinct basins separated by barriers that continuous exact-oracle methods could not cross.

2. **Exact oracles beat approximate gradients for local refinement.** When feasibility can be evaluated exactly in ∼0.5 ms, a greedy hill-climber using the exact oracle outperforms L-BFGS-B with approximate phi-function energy. The phi-function's Φ_{PP} component is unreliable for non-anti-parallel flat faces, distorting the optimisation landscape.

3. **Approximate inner loops enable basin exploration.** The two largest score improvements ($\Delta R = 0.013$ via LNS, $\Delta R = 0.011$ via fast approximate MCMC) both required methods that could modify the contact graph or explore at speeds impossible for exact-oracle methods. Exact refinement alone, applied to either basin, could not reach the other.

4. **The approximate/exact gap is a search signal.** Configurations with promising approximate scores that fail exact validation represent near-misses on the validity boundary. Retaining these and using them as restart seeds improved search diversity in the fast MCMC phase.

5. **No conjugate pairs in the best solution.** Zero antiparallel flat-face contacts appear in the best solution, despite the intuition that pairing flat faces would improve local density. For $N = 15$, the optimal regime appears to avoid flat-face pair formation.

Open questions. What is the true optimum for $N = 15$ (likely in $[2.85, 2.95]$ based on area bounds and observed landscape structure)? Does the global optimum use a ring topology, and if so which one? The 3-11-1 and 4-11 topologies are structurally distinct basins; whether a third, undiscovered topology exists below 2.948 is unknown. Can the phi-function be corrected for bounded semicircular segments to enable faster gradient-based polishing?

Reproducibility. Code is available at `https://github.com/CamArmstr/shape-packing-challenge`. The best solution JSON (`best_solution.json`), solutions archive (`solutions/`), and all optimiser implementations are included. The best result $R = 2.948572$ corresponds to commit `e5eb8ba`. Hyperparameters for each optimiser are hardcoded in the respective scripts; no external configuration files are required.

Acknowledgements. Problem posed by benedictbrady via the Optimization Arena [Optimization Arena, 2026]. The fast approximate MCMC approach (Section 3.8) builds on compusophy's parallel tempering architecture [compusophy, 2026], as detailed in Section 3.8. Computational work performed on a Windows 11 / WSL2 machine with a 16-core Intel CPU.

References

Bernardetta Addis, Marco Locatelli, and Fabio Schoen. Efficiently packing unequal disks in a circle. *Operations Research Letters*, 36(1):37–42, 2008. doi: 10.1016/j.orl.2007.03.001.

Peter Brass, William Moser, and János Pach. *Research Problems in Discrete Geometry*. Springer, New York, 2005. Section 1.5: semicircle packing listed as open.

Nikolai Chernov, Yuriy Stoyan, Tatiana Romanova, and Alexander Pankratov. Phi-functions for 2D objects formed by line segments and circular arcs. *Advances in Operations Research*, 2012: Article ID 346358, 2012. doi: 10.1155/2012/346358.

compusophy. pack-semicircles: Browser-based parallel tempering MCMC for semicircle packing. `https://github.com/compusophy/pack-semicircles`, 2026. Client-side React/TypeScript optimizer using 12 explorers + 4 greedy polishers; best $R = 2.94960$.

László Fejes Tóth. Packing and covering. In *Handbook of Convex Geometry*. North-Holland, 1971. Semicircle packing density question posed; still open.

Andrea Grosso, Marco Locatelli, and Fabio Schoen. Solving the problem of packing equal and unequal circles in a circular container. *Journal of Global Optimization*, 47(1):63–86, 2010. doi: 10.1007/s10898-009-9458-3.

Optimization Arena. Shape packing challenge: Semicircles. `https://optimization-arena.com/packing`, 2026. Competition platform; scoring rounds coordinates to 6 decimal places, validates via Shapely polygon intersection.

Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming (CP-98)*, pages 417–431. Springer, 1998. Original large neighbourhood search formulation.

Eckard Specht. Packomania: The encyclopedia of circle packings. `http://packomania.com`, 2024. Contains circles-in-circle data; no semicircle-in-circle results.